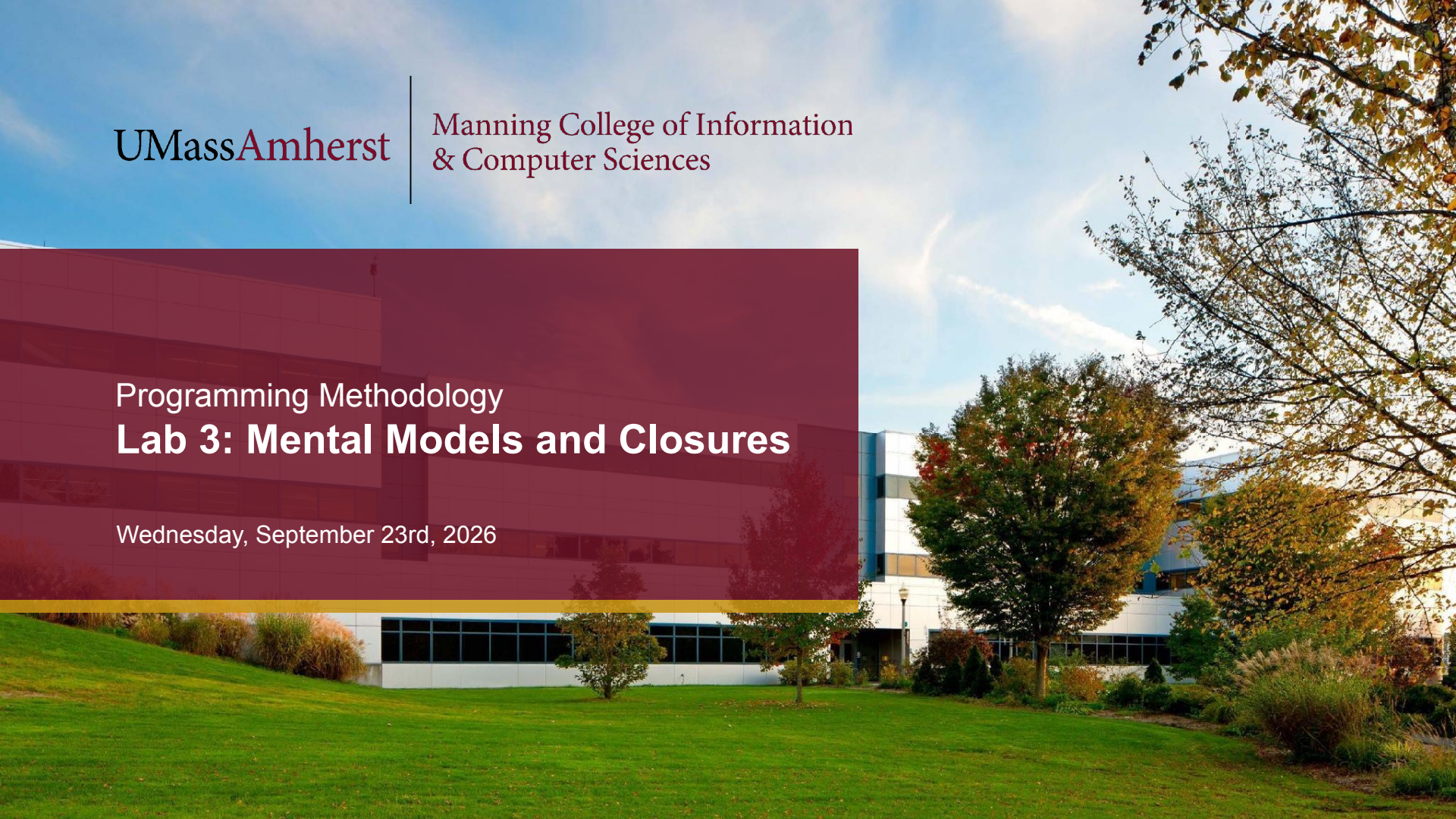


The background of the slide is a photograph of a modern, multi-story building with large windows and a flat roof. In the foreground, there is a well-maintained green lawn. Several trees with autumn-colored leaves (yellows, oranges, and browns) are scattered around the building and in the foreground. The sky is blue with some light clouds.

UMassAmherst

Manning College of Information
& Computer Sciences

Programming Methodology

Lab 3: Mental Models and Closures

Wednesday, September 23rd, 2026

Weekly Lab Agenda

- Go over reminders/goals
- Review past material
- Work in groups of 3-4 to solve a few exercises.
 - Please sit with your group from last week.
- Discussion leaders will walk around and answer questions.
- Solutions to exercises will be reviewed as a class.

Reminders

- Homework 3 is due next Monday (9/28) at 11:59pm.
 - Come to office hours for help!
- If you need to miss lab for a valid and documented reason:
 - Fill out the excusal form on Canvas before the lab.
 - Waking up late or the bus being late are not excusable reasons.
- Submit what you have at the end of lab to Gradescope.
 - Lab submissions are due at 6:00 pm.
 - The submission counts towards your lab grade.
 - **This lab includes a worksheet as well as a coding exercise.**
 - **The worksheet will be graded for completion.**

Today's Goals

- Practice using mental models
- Practice using closures

Review of Mental Models

A **mental model** represents our understanding of programming language semantics. In JavaScript and TypeScript,

- Primitive type semantics are **unspecified**.
 - Different JavaScript interpreters could use value or reference semantics.
- Objects (including arrays and functions) follow **reference semantics**.
- Function arguments are passed by value (**call-by-value**).

The stack **frame** represents an execution context and contains local variables, some of which may be references to **objects**.

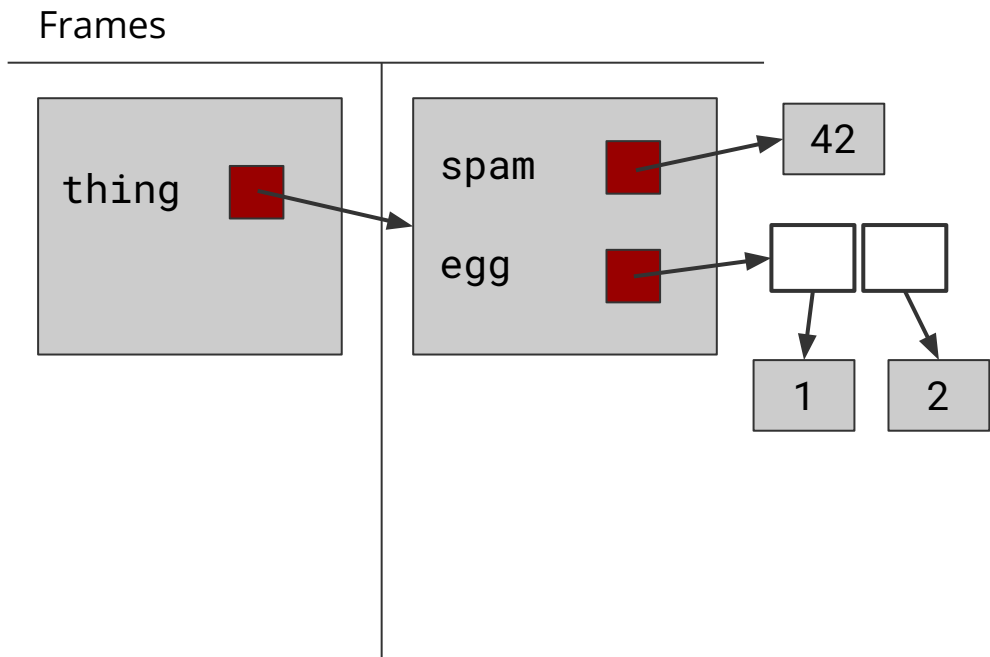
- Objects are stored separately from stack frames.
- An object stays in memory as long as there is a reference to it (from any frame).

Exercise 1: Mental Models

```
1 let thing = {  
2   spam: 42,  
3   egg: [1, 2]  
4 };
```

The memory diagram for the given code is depicted on the right.

It uses reference semantics for all types.

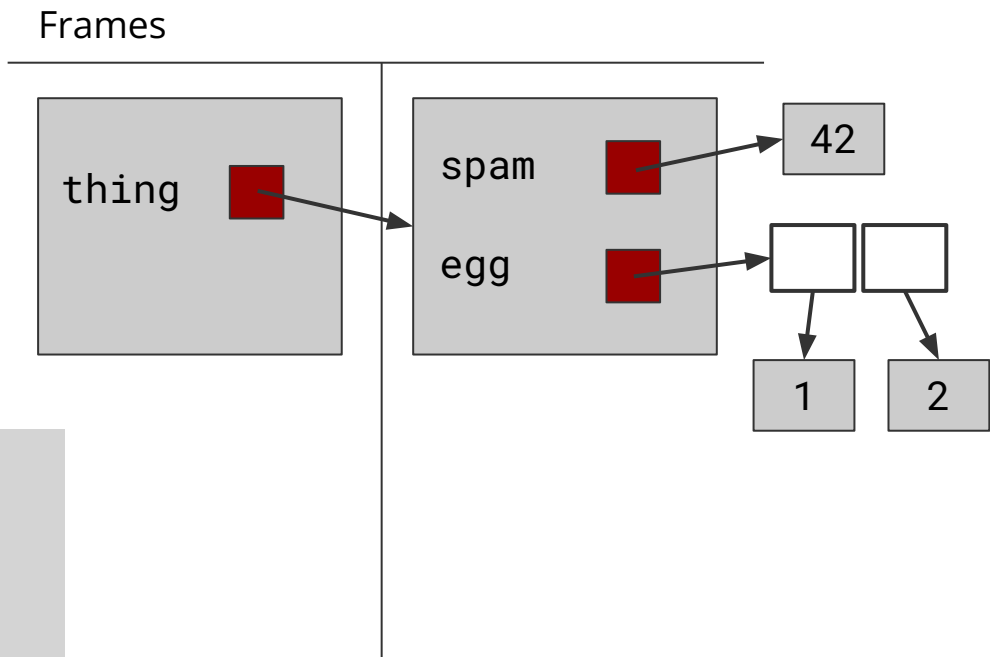


Exercise 1: Mental Models

```
1 let thing = {  
2   spam: 42,  
3   egg: [1, 2]  
4 };
```

Using the provided worksheet, update the memory diagram for each line of code below. Redraw any persistent elements from previous steps.

```
5 let tmp = thing.egg;  
6 thing.egg[0] += 3;  
7 thing.egg = thing.spam;  
8 thing.spam = tmp;  
9 thing.spam.push(thing.egg);  
10 thing.spam.filter(x => x < 5);
```

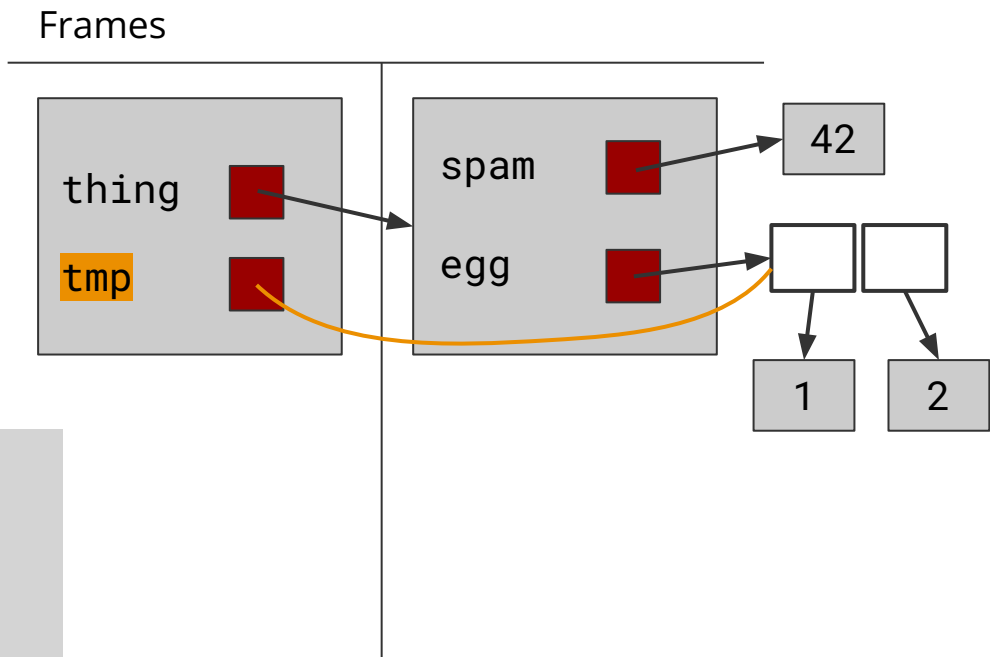


Solution 1: Mental Models

Line 5:

- variable `tmp` is initialized
- the value stored in `thing.egg` (reference to array `[1,2]`) is stored in `tmp`

```
5 let tmp = thing.egg;  
6 thing.egg[0] += 3;  
7 thing.egg = thing.spam;  
8 thing.spam = tmp;  
9 thing.spam.push(thing.egg);  
10 thing.spam.filter(x => x < 5);
```

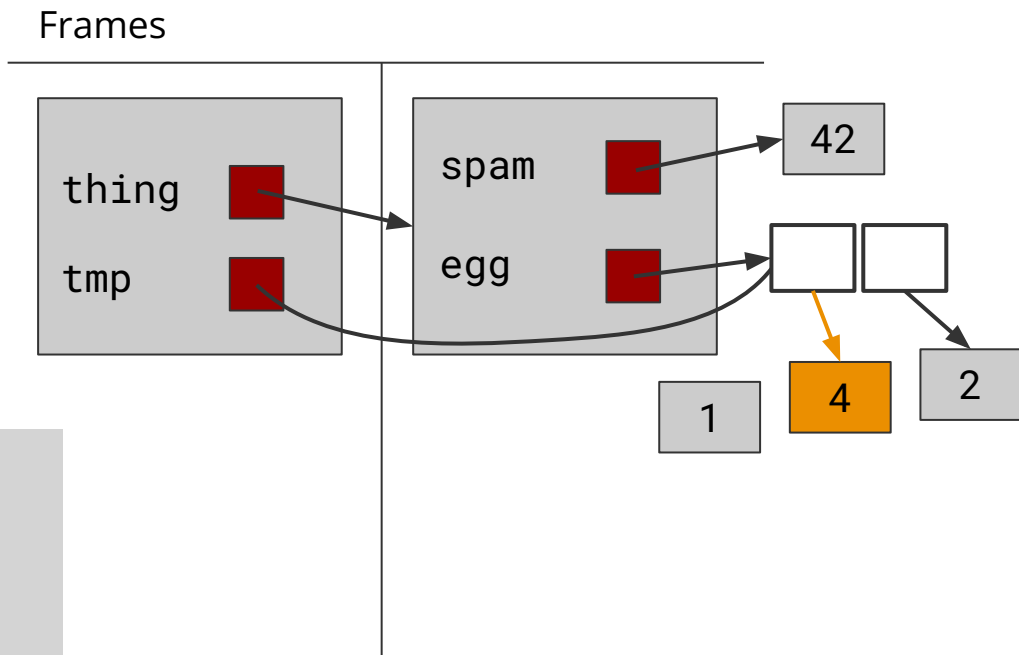


Solution 1: Mental Models

Line 6:

- the element at index 0 in the array referenced by `thing.egg` is incremented by 3

```
5 let tmp = thing.egg;  
6 thing.egg[0] += 3;  
7 thing.egg = thing.spam;  
8 thing.spam = tmp;  
9 thing.spam.push(thing.egg);  
10 thing.spam.filter(x => x < 5);
```

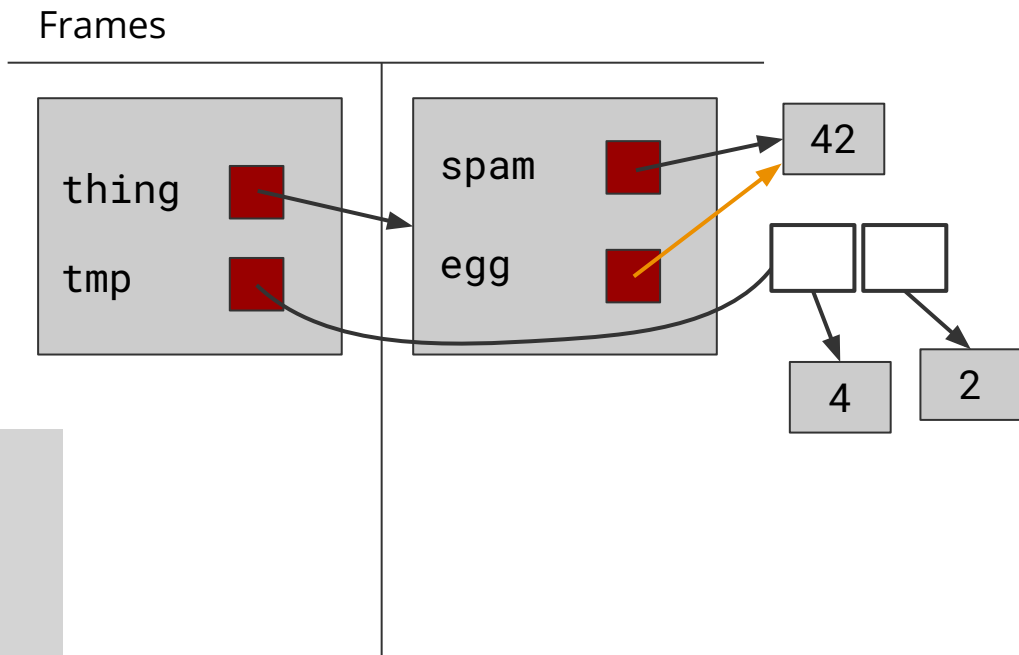


Solution 1: Mental Models

Line 7:

- the egg property of object thing is set to the value stored in thing.spam, which is the number 42

```
5 let tmp = thing.egg;  
6 thing.egg[0] += 3;  
7 thing.egg = thing.spam;  
8 thing.spam = tmp;  
9 thing.spam.push(thing.egg);  
10 thing.spam.filter(x => x < 5);
```



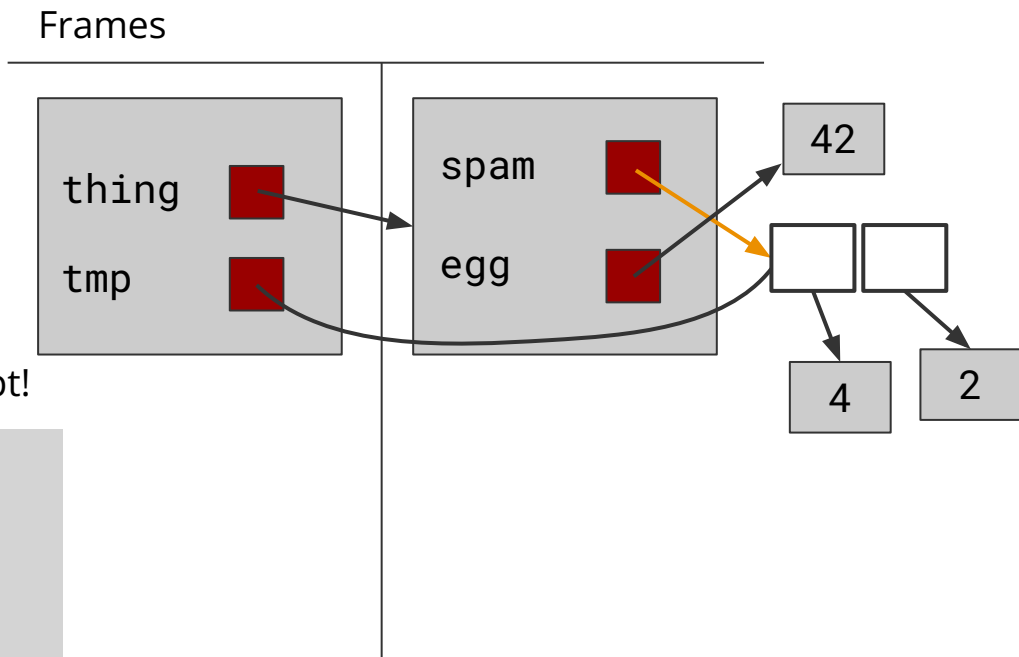
Solution 1: Mental Models

Line 8:

- the spam property of object thing is set to the value stored in tmp, which is the reference to the array [4, 2]

Note: lines 7 and 8 would error in Typescript!

```
5 let tmp = thing.egg;  
6 thing.egg[0] += 3;  
7 thing.egg = thing.spam;  
8 thing.spam = tmp;  
9 thing.spam.push(thing.egg);  
10 thing.spam.filter(x => x < 5);
```

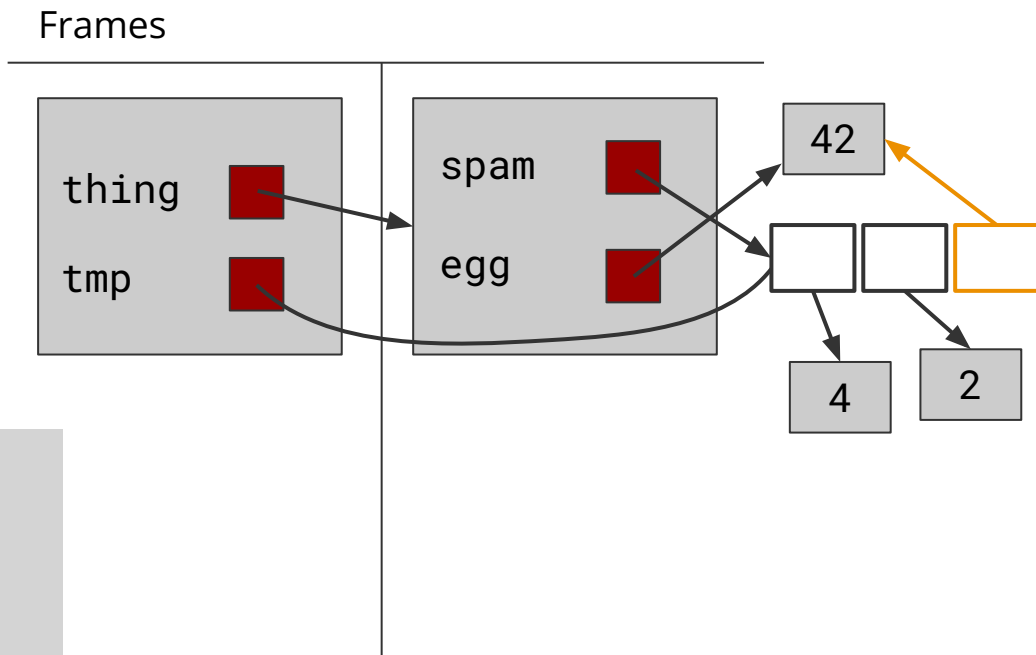


Solution 1: Mental Models

Line 9:

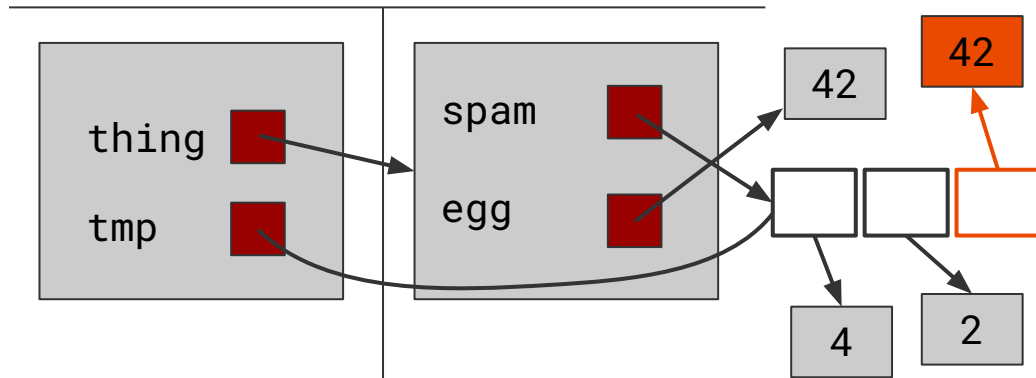
- the value stored in `thing.egg` (42) is added as an element to the array referenced by `thing.spam`

```
5 let tmp = thing.egg;  
6 thing.egg[0] += 3;  
7 thing.egg = thing.spam;  
8 thing.spam = tmp;  
9 thing.spam.push(thing.egg);  
10 thing.spam.filter(x => x < 5);
```



Solution 1: Mental Models

Frames



```
5 let tmp = thing.egg;
6 thing.egg[0] += 3;
7 thing.egg = thing.spam;
8 thing.spam = tmp;
9 thing.spam.push(thing.egg);
10 thing.spam.filter(x => x < 5);
```

Why is it **incorrect** for `thing.spam[2]` to point to a different 42?

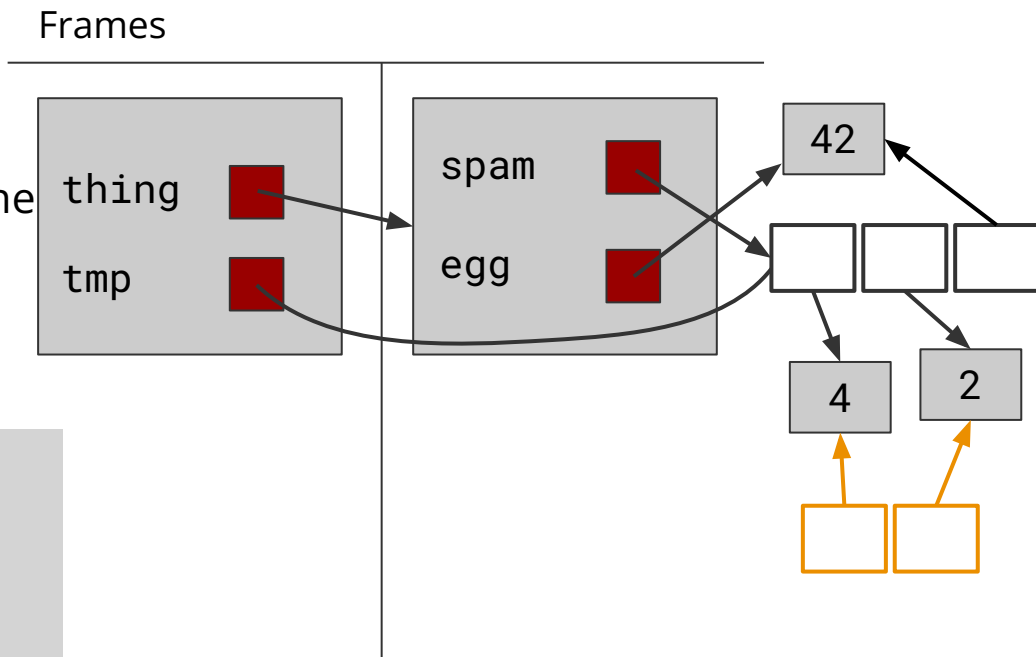
`thing.spam.push(thing.egg)` means we are adding a **new element** to the array referenced by `thing.spam`, where this element points to the same value as `thing.egg`. So, it should point to the **same 42** as `thing.egg`.

Solution 1: Mental Models

Line 10:

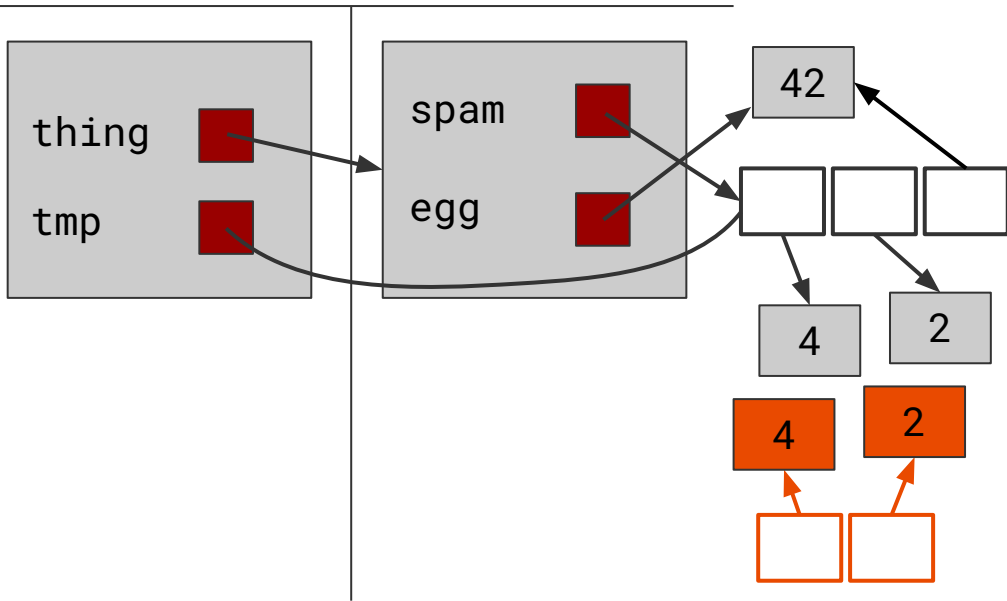
- `filter` is called on the array referenced by `thing.spam` (`[4, 2, 42]`)
- a new array is created, with only the elements less than 5 (`[4, 2]`)
 - the reference to this array is never stored

```
5 let tmp = thing.egg;  
6 thing.egg[0] += 3;  
7 thing.egg = thing.spam;  
8 thing.spam = tmp;  
9 thing.spam.push(thing.egg);  
10 thing.spam.filter(x => x < 5);
```



Solution 1: Mental Models

Frames



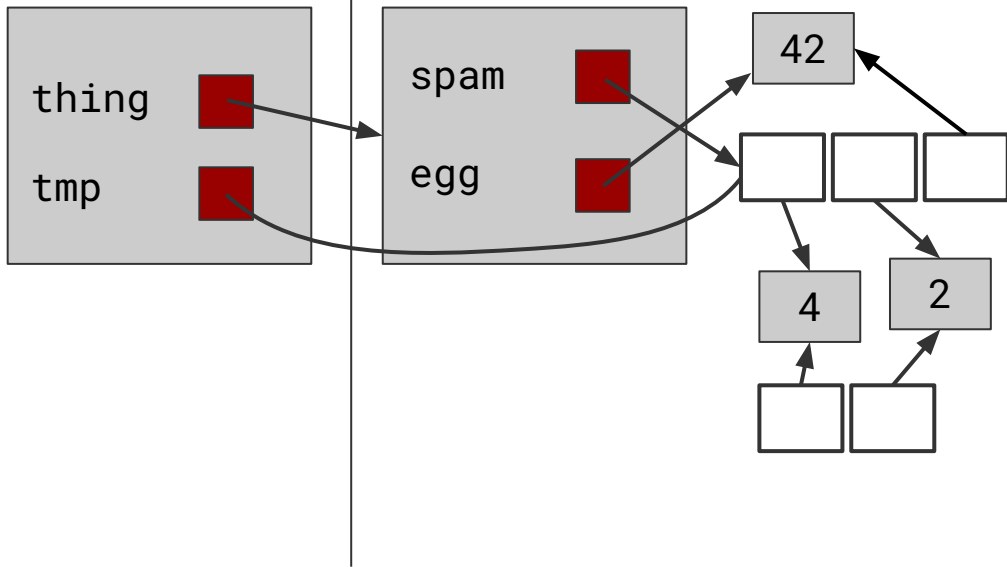
Why is it **incorrect** for the elements of the new array to point to different numbers than those pointed to by the existing array?

filter creates a shallow copy, so the elements of the new array will point to the same numbers as the original array.

```
5 let tmp = thing.egg;
6 thing.egg[0] += 3;
7 thing.egg = thing.spam;
8 thing.spam = tmp;
9 thing.spam.push(thing.egg);
10 thing.spam.filter(x => x < 5);
```

Solution 1: Mental Models

Frames

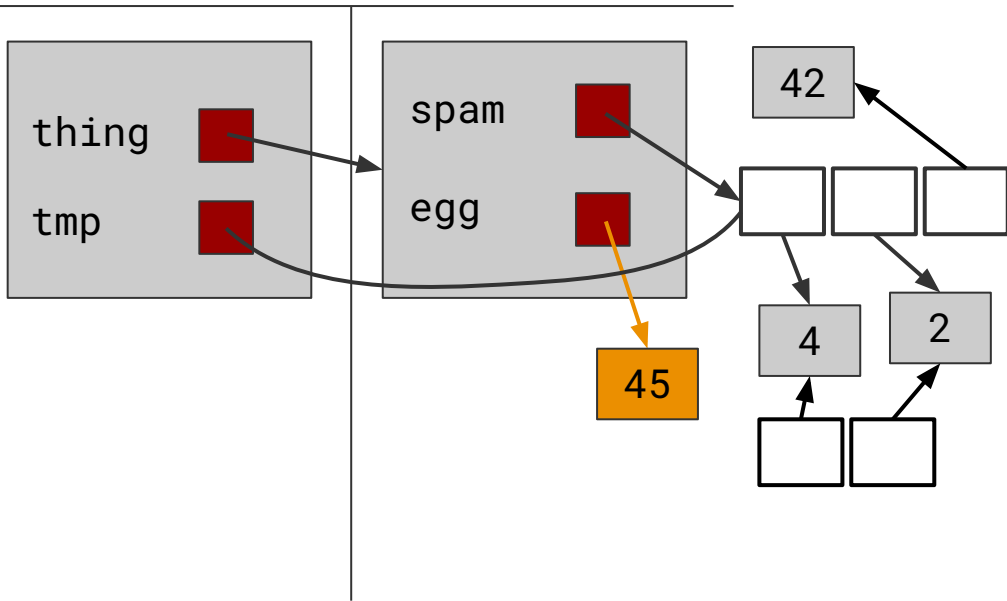


What would happen if we now ran the following lines of code?

```
11 thing.egg += 3;  
12 thing.spam[0] += 3;  
13 tmp[1] += 3;
```

Solution 1: Mental Models

Frames



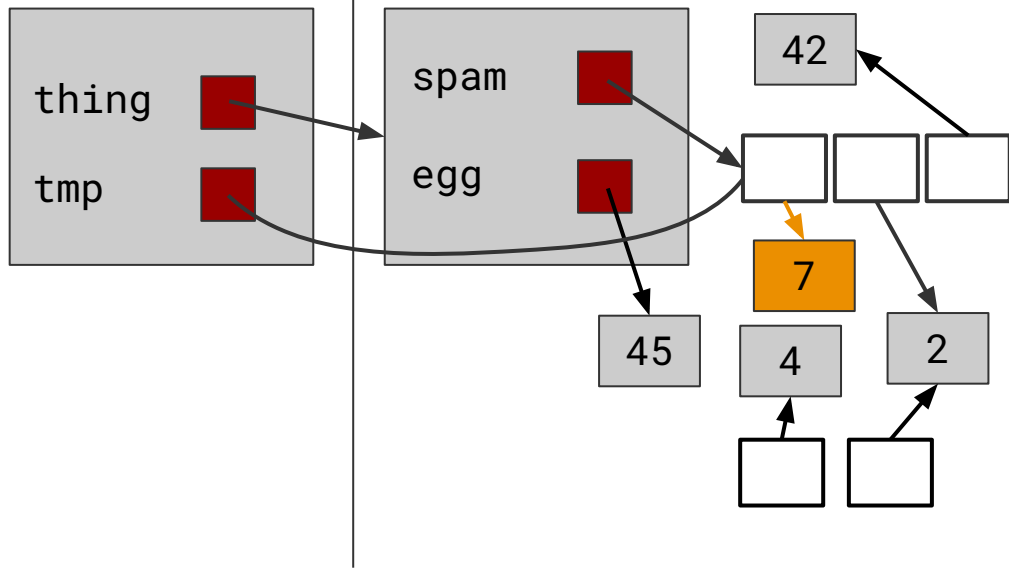
Line 11:

- we increment `thing.egg` by 3, meaning `thing.egg` now points to a different number (45)
- the 42 that `thing.egg` originally pointed to is not modified to become 45, as numbers are immutable

```
11 thing.egg += 3;  
12 thing.spam[0] += 3;  
13 tmp[1] += 3;
```

Solution 1: Mental Models

Frames



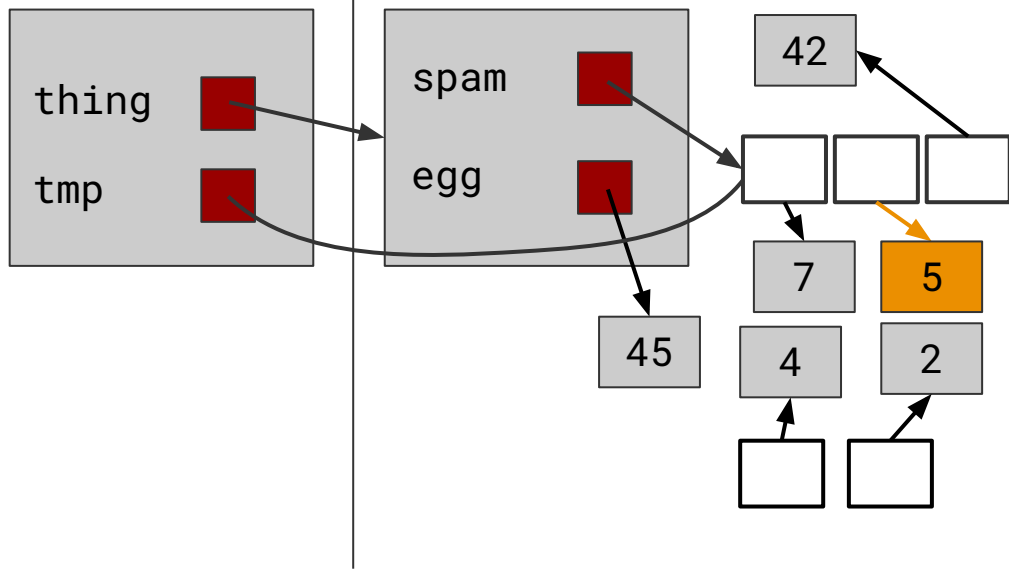
Line 12:

- we increment the first element of the array referenced by `thing.spam` by 3
- now, `thing.spam[0]` points to 7

```
11 thing.egg += 3;  
12 thing.spam[0] += 3;  
13 tmp[1] += 3;
```

Solution 1: Mental Models

Frames



Line 13:

- we increment the second element of the array referenced by tmp by 3
- now, tmp[1] points to 5

```
11 thing.egg += 3;  
12 thing.spam[0] += 3;  
13 tmp[1] += 3;
```

Review of Closures

A closure is a **function** bundled with its **surrounding environment**.

Closures can be used to **hide state** and dictate **specialized interaction**.

In JavaScript and TypeScript,

- A closure is created every time a function is created.
- Functions are **first-class**, so they can be used like any other value.
 - Unnamed functions are called **anonymous** functions.

Exercise 2: Closures

Write a function **factorialSum** that takes no arguments.

It should return a closure that, when called the n^{th} time ($n \geq 1$), returns the sum of factorials up to n

$$f(n) = 1! + 2! + \dots + n!$$

Avoid unnecessary recomputation.

Example outputs: 1, 3, 9, 33, 153, ...

Solution 2: Closures

```
function factorialSum(): () => number {  
  let n = 1, factorial = 1, sum = 0;  
  // can you shorten this even further?  
  
  return () => {  
    factorial *= n++; // why n++ and not ++n?  
    sum += factorial;  
    return sum;  
  }  
}
```

Recap

	Mental Models	Closures
Purpose	Help us reason about what a program is doing	Allow a function to use variables from its surrounding scope, even after that scope has finished executing
Core idea	Represent our understanding of program language semantics	Combines a function and its surrounding environment
Key concepts	State, stack frames, values, references, garbage collection	Scope, persistence, state hiding
Takeaway	Use mental models throughout development to reason about code	Use closures to implement persistent state and encapsulation